Cross-platform development

Flutter

Dart

Introduction to the language

Language particularities

Dart

https://dart.dev/guides/language/language-tour

- In Flutter the development language is Dart
  - Language developed by Google
  - Initially used only internally, for a small set of applications
  - Later it start to be used for Flutter application development
- In addition to Dart, it may be necessary to use the native languages of the platform to be used
  - For example, if it is necessary to create something that is not yet available in Dart for Flutter, Kotlin in Android, Swift in iOS or other platform specific language must be used
- Like Kotlin, it has a *DartPad* that can be used for small language tests:
  - https://dart.dev/#try-dart

Dart

- Dart is a single-threaded language, however it offers an API rich in asynchronous operations, similar to what happens with Javascript (parallelism ≠ concurrency)

- As in Kotlin, it offers support for several functions that make the programmer's life easier, including: *null safety*, *late types*, *spread operator*, *auto inference*, etc.

- It shares semantics with C-based languages, also resembling some Web languages (for example, Javascript and Typescript)

- It offers versatility as it can run “everywhere”, whether on a *mobile client*, *web*, *desktop*, server or embedded systems

Dart

- Everything that is placed in a variable is considered an object and all objects are class instances
  - Primary types such as numbers, functions or even `null` are also included
  - All objects derive directly or indirectly from `Object`
- Although Dart is strongly “typed”, it is not necessary to put types when declaring variables, as these can be inferred

Dart

- When using *null-safety* (default), variables cannot contain the `null` value – similar to Kotlin
  - For example, a `String` will always have a value, while a `String?` may have `null`
  - To unwrap *,* a *bang operator* (`!`) must be used

- When it is needed to have a variable that can receive objects of any type, the `dynamic` keyword must be used

Dart

- Dart supports generic types, like other languages
  - For example, a `List<int>` will only support a list of integers while a `List<Object>` will accept any type of object

- There is support for *top level functions and variables* which means they do not need to be inside classes – similar to Kotlin

Dart

- The `public`, `protected` and `private` modifiers don't exist in Dart
  - For any variable or method to be private to the class or file in which it is inserted, the _ character must be used

- The Dart tools can be used to display information about the app execution, as also warnings and errors

Basic Dart program

```dart
// Define a function.
void printInteger(int aNumber) {
  print('The number is $aNumber.'); // Print to console.
}

// This is where the app starts executing.
void main() {
  var number = 42; // Declare and initialize a variable.
  printInteger(number); // Call a function.
}
```

Variables

```
var name = 'Bob';
```

```
Object name = 'Bob';
```

```
String name = 'Bob';
```

```
late String description;

void main() {
  description = 'Feijoada!';
  print(description);
}
```

```
// This is the program's only call to readThermometer().
late String temperature = readThermometer(); // Lazily initialized.
```

final and const

```dart
final name = 'Bob'; // Without a type annotation
final String nickname = 'Bobby';
```

```
✗ static analysis: error/warning
```

```dart
name = 'Alice'; // Error: a final variable can only be set once.
```

```dart
const bar = 1000000; // Unit of pressure (dynes/cm2)
const double atm = 1.01325 * bar; // Standard atmosphere
```

Note: The constants are defined in *compile time*, making them more efficient and should be used whenever possible

Supported types

- Main
  - Numbers (`int`, `double`)
  - Strings (`String`)
  - Booleans (`bool`)
  - Lists (`List`, also known as arrays)
  - Sets (`Set`)
  - Maps (Map)
  - Runes (`Runes`; often replaced by the `characters` API)
  - Symbols (`Symbol`)
  - The value `null` (`Null`)

Supported types

- Others
  - `Object`
    - The superclass of all Dart classes except `Null`
  - `Enum`
    - The superclass of all enums
  - `Future` and `Stream`
    - Used in asynchronous support
  - `Iterable`
    - Used in for-in loops and in synchronous generator functions
  - `Never`
    - Indicates that an expression can never successfully finish evaluating. Most often used for functions that always throw an exception
  - `dynamic`
    - Indicates that static checking must be disabled. Usually `Object` or `Object?` should be used instead
  - `void`
    - Indicates that a value is never used. Often used as a return type

Data types

```
// String -> int
var one = int.parse('1');
assert(one == 1);

// String -> double
var onePointOne = double.parse('1.1');
assert(onePointOne == 1.1);

// int -> String
String oneAsString = 1.toString();
assert(oneAsString == '1');

// double -> String
String piAsString = 3.14159.toStringAsFixed(2);
assert(piAsString == '3.14');
```

```
var s = 'string interpolation';

assert('Dart has $s, which is very handy.' ==
    'Dart has string interpolation, '
        'which is very handy.');
assert('That deserves all caps. '
        '${s.toUpperCase()} is very handy!' ==
    'That deserves all caps. '
        'STRING INTERPOLATION is very handy!');
```

```
assert((3 << 1) == 6); // 0011 << 1 == 0110
assert((3 | 4) == 7); // 0011 | 0100 == 0111
assert((3 & 4) == 0); // 0011 & 0100 == 0000
```

```
var s1 = 'Single quotes work well for string literals.';
var s2 = "Double quotes work just as well.";
var s3 = 'It\'s easy to escape the string delimiter.';
var s4 = "It's even easier to use the other delimiter.";
```

```
var s1 = '''
You can create
multi-line strings like this one.
''';

var s2 = """This is also a
multi-line string.""";
```

```
// These work in a const string.
const aConstNum = 0;
const aConstBool = true;
const aConstString = 'a constant string';

// These do NOT work in a const string.
var aNum = 0;
var aBool = true;
var aString = 'a string';
const aConstList = [1, 2, 3];

const validConstString = '$aConstNum $aConstBool $aConstString';
// const invalidConstString = '$aNum $aBool $aString $aConstList';
```

Collections and spread operators

```
const Object i = 3; // Where i is a const Object with an int value...
const list = [i as int]; // Use a typecast.
const map = {if (i is int) i: 'int'}; // Use is and collection if.
const set = {if (list is List<int>) ...list}; // ...and a spread.
```

```
final list = <int>[];
list.add(1);
list.add(2);
```

```
var foo = const [];
final bar = const [];
const baz = []; // Equivalent to `const []`

foo = [1, 2, 3]; // Was const []
```

```
var list = [
  'Car',
  'Boat',
  'Plane',
];
```

```
final constantSet = const {
  'fluorine',
  'chlorine',
  'bromine',
  'iodine',
  'astatine',
};
```

```
var halogens = {'fluorine', 'chlorine', 'bromine', 'iodine', 'astatine'};
```

```
var gifts = Map<String, String>();
gifts['first'] = 'partridge';
gifts['second'] = 'turtledoves';
gifts['fifth'] = 'golden rings';

var nobleGases = Map<int, String>();
nobleGases[2] = 'helium';
nobleGases[10] = 'neon';
nobleGases[18] = 'argon';
```

```
var gifts = {
  // Key:    Value
  'first': 'partridge',
  'second': 'turtledoves',
  'fifth': 'golden rings'
};

var nobleGases = {
  2: 'helium',
  10: 'neon',
  18: 'argon',
};
```

```
final constantMap = const {
  2: 'helium',
  10: 'neon',
  18: 'argon',
};
```

Functions

```
bool isNoble(int atomicNumber) {
  return _nobleGases[atomicNumber] != null;
}
```

```
isNoble(atomicNumber) {
  return _nobleGases[atomicNumber] != null;
}
```

```
bool isNoble(int atomicNumber) => _nobleGases[atomicNumber] != null;
```

```
/// Sets the [bold] and [hidden] flags ...
void enableFlags({bool? bold, bool? hidden}) {...}

enableFlags(bold: true, hidden: false);
```

```
void printElement(int element) {
  print(element);
}

var list = [1, 2, 3];

// Pass printElement as a parameter.
list.forEach(printElement);
```

```
String say(String from, String msg, [String? device]) {
  var result = '$from says $msg';
  if (device != null) {
    result = '$result with a $device';
  }
  return result;
}

assert(say('Bob', 'Howdy') == 'Bob says Howdy');
assert(say('Bob', 'Howdy', 'smoke signal') ==
    'Bob says Howdy with a smoke signal');
```

```
var loudify = (msg) => '!!! ${msg.toUpperCase()} !!!';
assert(loudify('hello') == '!!! HELLO !!!');
```

```
const list = ['apples', 'bananas', 'oranges'];
list.map((item) {
  return item.toUpperCase();
}).forEach((item) {
  print('$item: ${item.length}');
});
```

Operators

Description	Operator	Associativity
unary postfix	*expr++* *expr--* () [] ?[] . ?. !	None
unary prefix	*-expr* *!expr* *~expr* *++expr* *--expr* await *expr*	None
multiplicative	* / % ~/	Left
additive	+ -	Left
shift	<< >> >>>	Left
bitwise AND	&	Left
bitwise XOR	^	Left
bitwise OR	|	Left
relational and type test	>= > <= < as is is!	None
equality	== !=	None
logical AND	&&	Left
logical OR	||	Left
if null	??	Left
conditional	*expr1* ? *expr2* : *expr3*	Right
cascade	.. ?..	Right
assignment	= *= /= += -= &= ^= *etc.*	Right

```
(employee as Person).firstName = 'Bob';
```

```
if (employee is Person) {
  // Type check
  employee.firstName = 'Bob';
}
```

```
var paint = Paint()
  ..color = Colors.black
  ..strokeCap = StrokeCap.round
  ..strokeWidth = 5.0;
```

```
// Assign value to b if b is null; otherwise, b stays the same
b ??= value;
```

```
String playerName(String? name) => name ?? 'Guest';
```

if-else and assert

```
if (isRaining()) {
  you.bringRainCoat();
} else if (isSnowing()) {
  you.wearJacket();
} else {
  car.putTopDown();
}
```

```
// Make sure the variable has a non-null value.
assert(text != null);

// Make sure the value is less than 100.
assert(number < 100);

// Make sure this is an https URL.
assert(urlString.startsWith('https'));
```

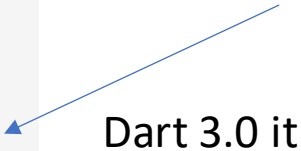
```
assert(urlString.startsWith('https'),
    'URL ($urlString) should start with "https".');
```

switch-case

```dart
var command = 'OPEN';
switch (command) {
  case 'CLOSED':
    executeClosed();
    break;
  case 'PENDING':
    executePending();
    break;
  case 'APPROVED':
    executeApproved();
    break;
  case 'DENIED':
    executeDenied();
    break;
  case 'OPEN':
    executeOpen();
    break;
  default:
    executeUnknown();
}
```

```dart
var command = 'OPEN';
switch (command) {
  case 'OPEN':
    executeOpen();
    // ERROR: Missing break

  case 'CLOSED':
    executeClosed();
    break;
}
```



Dart 3.0 it is allowed

```dart
var command = 'CLOSED';
switch (command) {
  case 'CLOSED': // Empty case falls through.
  case 'NOW_CLOSED':
    // Runs for both CLOSED and NOW_CLOSED.
    executeNowClosed();
    break;
}
```

for, while and do... while

```
var message = StringBuffer('Dart is fun');
for (var i = 0; i < 5; i++) {
  message.write('!');
}
```

```
var collection = [1, 2, 3];
collection.forEach(print); // 1 2 3
```

```
var callbacks = [];
for (var i = 0; i < 2; i++) {
  callbacks.add(() => print(i));
}

for (final c in callbacks) {
  c();
}
```

```
while (!isDone()) {
  doSomething();
}
```

```
do {
  printLine();
} while (!atEndOfPage());
```

try-catch-finally , throw

```
try {
  breedMoreLlamas();
} on OutOfLlamasException {
  buyMoreLlamas();
}
```

```
try {
  breedMoreLlamas();
} on OutOfLlamasException {
  // A specific exception
  buyMoreLlamas();
} on Exception catch (e) {
  // Anything else that is an exception
  print('Unknown exception: $e');
} catch (e) {
  // No specified type, handles all
  print('Something really unknown: $e');
}
```

```
throw FormatException('Expected at least 1 section');
```

```
void misbehave() {
  try {
    dynamic foo = true;
    print(foo++); // Runtime error
  } catch (e) {
    print('misbehave() partially handled ${e.runtimeType}.');
    rethrow; // Allow callers to see the exception.
  }
}

void main() {
  try {
    misbehave();
  } catch (e) {
    print('main() finished handling ${e.runtimeType}.');
  }
}
```

```
try {
  breedMoreLlamas();
} catch (e) {
  print('Error: $e'); // Handle the exception first.
} finally {
  cleanLlamaStalls(); // Then clean up.
}
```

Classes

```
const double xOrigin = 0;
const double yOrigin = 0;

class Point {
  final double x;
  final double y;

  Point(this.x, this.y);

  // Named constructor
  Point.origin()
      : x = xOrigin,
        y = yOrigin;
}
```

```
var p1 = Point(2, 2);
var p2 = Point.fromJson({'x': 1, 'y': 2});
```

```
var p1 = new Point(2, 2);
var p2 = new Point.fromJson({'x': 1, 'y': 2});
```

```
class Point {
  double? x; // Declare instance variable x, initially null.
  double? y; // Declare y, initially null.
  double z = 0; // Declare z, initially 0.
}
```

```
class Employee extends Person {
  Employee() : super.fromJson(fetchDefaultData());
  // ···
}
```

```
class Vector2d {
  final double x;
  final double y;

  Vector2d(this.x, this.y);
}

class Vector3d extends Vector2d {
  final double z;

  // Forward the x and y parameters to the default super constructor like:
  // Vector3d(final double x, final double y, this.z) : super(x, y);
  Vector3d(super.x, super.y, this.z);
}
```

Classes

```dart
class Point {
  double x, y;

  // The main constructor for this class.
  Point(this.x, this.y);

  // Delegates to the main constructor.
  Point.alongXAxis(double x) : this(x, 0);
}
```

```dart
class Logger {
  final String name;
  bool mute = false;

  // _cache is library-private, thanks to
  // the _ in front of its name.
  static final Map<String, Logger> _cache = <String, Logger>{};

  factory Logger(String name) {
    return _cache.putIfAbsent(name, () => Logger._internal(name));
  }

  factory Logger.fromJson(Map<String, Object> json) {
    return Logger(json['name'].toString());
  }

  Logger._internal(this.name);

  void log(String msg) {
    if (!mute) print(msg);
  }
}
```

```dart
var logger = Logger('UI');
logger.log('Button clicked');

var logMap = {'name': 'UI'};
var loggerJson = Logger.fromJson(logMap);
```

```dart
class ImmutablePoint {
  static const ImmutablePoint origin = ImmutablePoint(0, 0);

  final double x, y;

  const ImmutablePoint(this.x, this.y);
}
```

Methods

```dart
import 'dart:math';

class Point {
  final double x;
  final double y;

  Point(this.x, this.y);

  double distanceTo(Point other) {
    var dx = x - other.x;
    var dy = y - other.y;
    return sqrt(dx * dx + dy * dy);
  }
}
```

```dart
class Television {
  // ···
  set contrast(int value) {...}
}

class SmartTelevision extends Television {
  @override
  set contrast(num value) {...}
  // ···
}
```

```dart
class Vector {
  final int x, y;

  Vector(this.x, this.y);

  Vector operator +(Vector v) => Vector(x + v.x, y + v.y);
  Vector operator -(Vector v) => Vector(x - v.x, y - v.y);

  @override
  bool operator ==(Object other) =>
      other is Vector && x == other.x && y == other.y;

  @override
  int get hashCode => Object.hash(x, y);
}

void main() {
  final v = Vector(2, 3);
  final w = Vector(2, 2);

  assert(v + w == Vector(4, 5));
  assert(v - w == Vector(0, 1));
}
```

Getters and Setters

```
class Rectangle {
  double left, top, width, height;

  Rectangle(this.left, this.top, this.width, this.height);

  // Define two calculated properties: right and bottom.
  double get right => left + width;
  set right(double value) => left = value - width;
  double get bottom => top + height;
  set bottom(double value) => top = value - height;
}

void main() {
  var rect = Rectangle(3, 4, 20, 15);
  assert(rect.left == 3);
  rect.right = 12;
  assert(rect.left == -8);
}
```

Abstract classes and methods

```
// This class is declared abstract and thus
// can't be instantiated.
abstract class AbstractContainer {
  // Define constructors, fields, methods...

  void updateChildren(); // Abstract method.
}
```

```
abstract class Doer {
  // Define instance variables and methods...

  void doSomething(); // Define an abstract method.
}

class EffectiveDoer extends Doer {
  void doSomething() {
    // Provide an implementation, so the method is not abstract here...
  }
}
```

Extension

```
extension AMovString on String {
  String amov(int i) => '*' * i + this + '*' * i;
}

void main() {
  print('Hello, World!'.amov(5));
}

// output: *****Hello, World!*****
```

Enumerations

```
enum Color { red, green, blue }
```

```
assert(Color.red.index == 0);
assert(Color.green.index == 1);
assert(Color.blue.index == 2);
```

```
final favoriteColor = Color.blue;
if (favoriteColor == Color.blue) {
  print('Your favorite color is blue!');
}
```

```
var aColor = Color.blue;

switch (aColor) {
  case Color.red:
    print('Red as roses!');
    break;
  case Color.green:
    print('Green as grass!');
    break;
  default: // Without this, you see a WARNING.
    print(aColor); // 'Color.blue'
}
```

```
enum Vehicle implements Comparable<Vehicle> {
  car(tires: 4, passengers: 5, carbonPerKilometer: 400),
  bus(tires: 6, passengers: 50, carbonPerKilometer: 800),
  bicycle(tires: 2, passengers: 1, carbonPerKilometer: 0);

  const Vehicle({
    required this.tires,
    required this.passengers,
    required this.carbonPerKilometer,
  });

  final int tires;
  final int passengers;
  final int carbonPerKilometer;

  int get carbonFootprint => (carbonPerKilometer / passengers).round();

  @override
  int compareTo(Vehicle other) => carbonFootprint - other.carbonFootprint;
}
```

Mixins

- A *Mixin* allows to create a set of variables and/or methods that can be coupled to a class.
- A class can use *multiple mixins*, but can only extend one class

```
class Musician extends Performer with Musical {
  // ...
}

class Maestro extends Person with Musical, Aggressive, Demented {
  Maestro(String maestroName) {
    name = maestroName;
    canConduct = true;
  }
}
```

```
class Musician {
  // ...
}
mixin MusicalPerformer on Musician {
  // ...
}
class SingerDancer extends Musician with MusicalPerformer {
  // ...
}
```

```
mixin Musical {
  bool canPlayPiano = false;
  bool canCompose = false;
  bool canConduct = false;

  void entertainMe() {
    if (canPlayPiano) {
      print('Playing piano');
    } else if (canConduct) {
      print('Waving hands');
    } else {
      print('Humming to self');
    }
  }
}
```

There are significant changes in Dart 3.0

Importing libraries

```dart
import 'dart:html';
```

```dart
import 'package:test/test.dart';
```

```dart
import 'package:lib1/lib1.dart';
import 'package:lib2/lib2.dart' as lib2;

// Uses Element from lib1.
Element element1 = Element();

// Uses Element from lib2.
lib2.Element element2 = lib2.Element();
```

```dart
// Import only foo.
import 'package:lib1/lib1.dart' show foo;

// Import all names EXCEPT foo.
import 'package:lib2/lib2.dart' hide foo;
```

Asynchronous programming

- Methods that should be executed asynchronously must be tagged with the `async` keyword
  - These methods must return a `Future<T>`
  - The method will be included in the Dart event-loop
  - Multiple `async` runs in the context of the same *main thread* (...in the *main isolate*!)

Asynchronous programming

- When using the `await` keyword, the execution of the block is "stopped" in this instruction, until the asynchronous call that was made returns
  - When using `await`, the method/block where it is integrated must be tagged as asynchronous

```
Future<void> checkVersion() async {
  var version = await lookUpVersion();
  // Do something with version
}
```

```
try {
  version = await lookUpVersion();
} catch (e) {
  // React to inability to look up the version
}
```

```
var entrypoint = await findEntryPoint();
var exitCode = await runExecutable(entrypoint, args);
await flushThenExit(exitCode);
```

```
void main() async {
  checkVersion();
  print('In main: version is ${await lookUpVersion()}');
}
```

Asynchronous programming

- Alternatively, the `then` keyword can be used, which invokes a *callback* when the return of the asynchronous method is completed, avoiding having to wait for this instruction or force the caller code block to be asynchronous

```
myFunc().then(processValue).catchError(handleError);
```

Streams

```dart
Stream<int> amovStream(int n) async* {
  int c = 0;
  while ( c < n ) {
    print('creating... [$c]');
    yield c * c;
    c++;
    await Future.delayed(Duration(seconds: 1));
  }
}

void main() async{
  print('AMov: Begin');
  await for(var c in amovStream(5) ) {
    print('value received: $c');
  }
  print('AMov: End');
}
```

Console

```
AMov: Begin
creating... [0]
value received: 0
creating... [1]
value received: 1
creating... [2]
value received: 4
creating... [3]
value received: 9
creating... [4]
value received: 16
AMov: End
```

Callable classes

```
class WannabeFunction {
  String call(String a, String b, String c) => '$a $b $c!';
}

var wf = WannabeFunction();
var out = wf('Hi', 'there,', 'gang');

void main() => print(out);
```

Console

```
Hi there, gang!
```

Flutter

Introduction

Installation

Example project

Simple state management

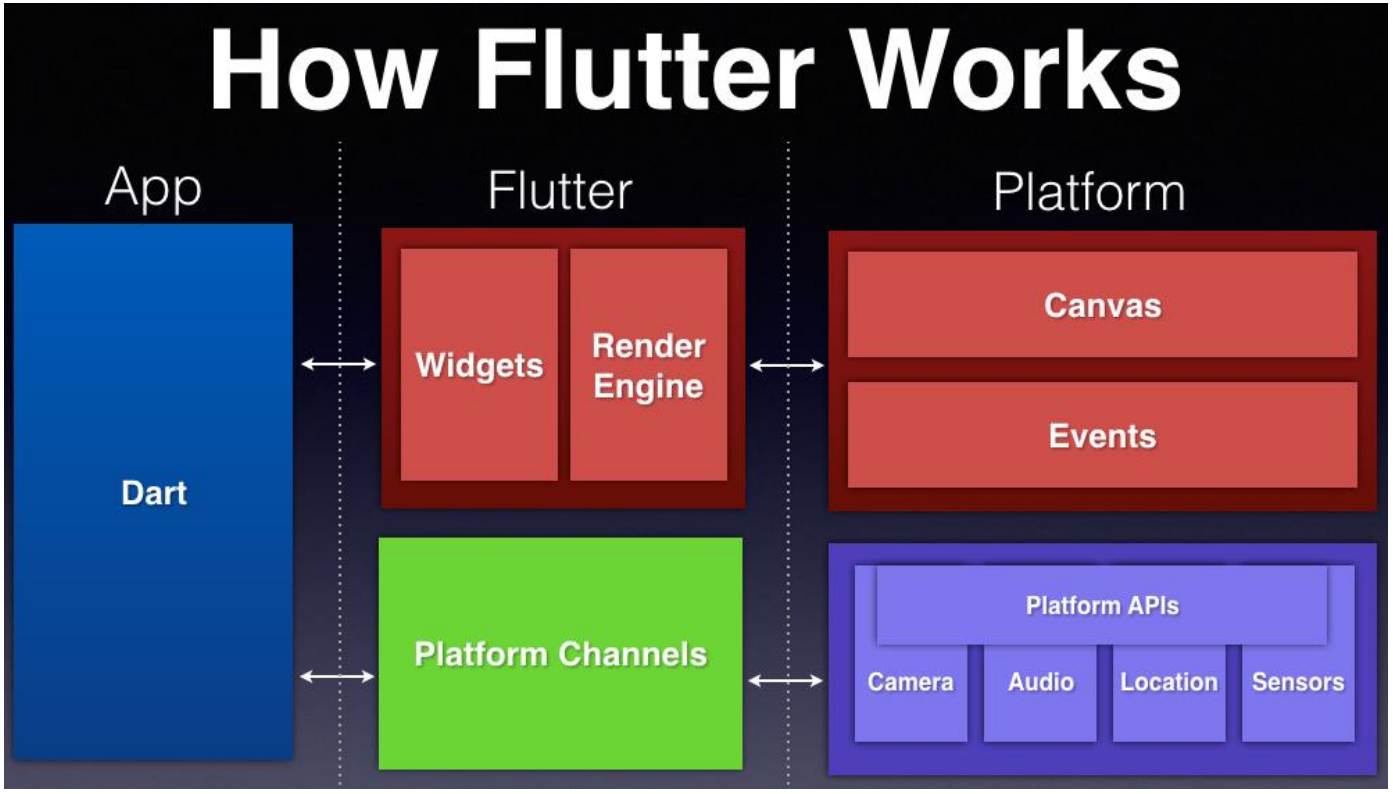
Introduction to Flutter

https://docs.flutter.dev

- **Flutter is what is usually called a Cross-platform**
  - Supports development for multiple platforms with the same code base (iOS, Android, Web, Desktop, etc.)
- A wide range of components (*widgets*) are available, which support different platform *guidelines* (*Material*, *Cupertino*, etc.)
- A declarative API is used for UI construction
- Support for various productive development tools like *hot-reload*, *hot-restart* , *layout inspect*, etc.
- Due to the use of Dart, it supports AoT and JIT compilation
- "*Allows excellent performance at 60+ fps*"

Flutter

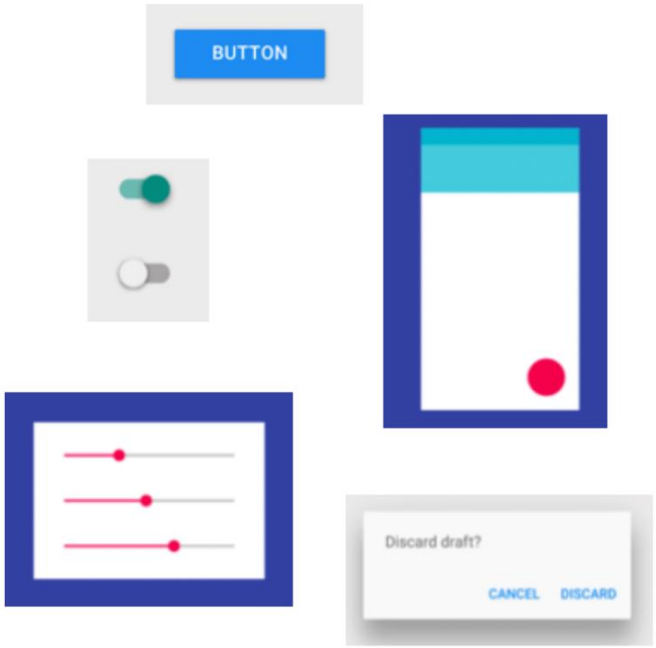
- Everything visible on the screen is a *widget*
- It is independent of OS versions and native API



https://medium.com/@hrcovertx/understanding-how-flutter-works-fdd54e3ca0d1

UI in Flutter

- Flutter follows the *guidelines* for Android and iOS platforms

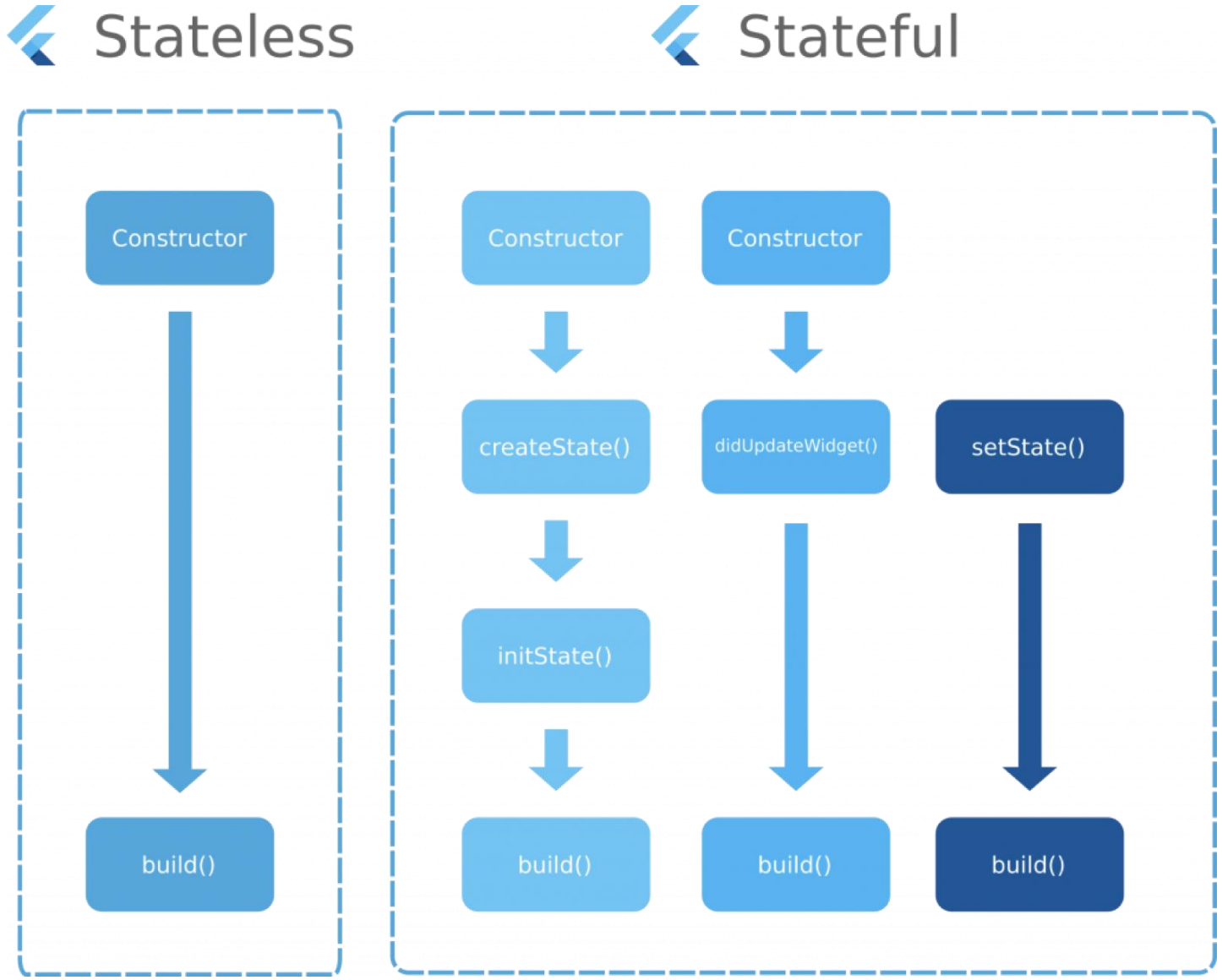


https://docs.flutter.dev/development/ui/widgets

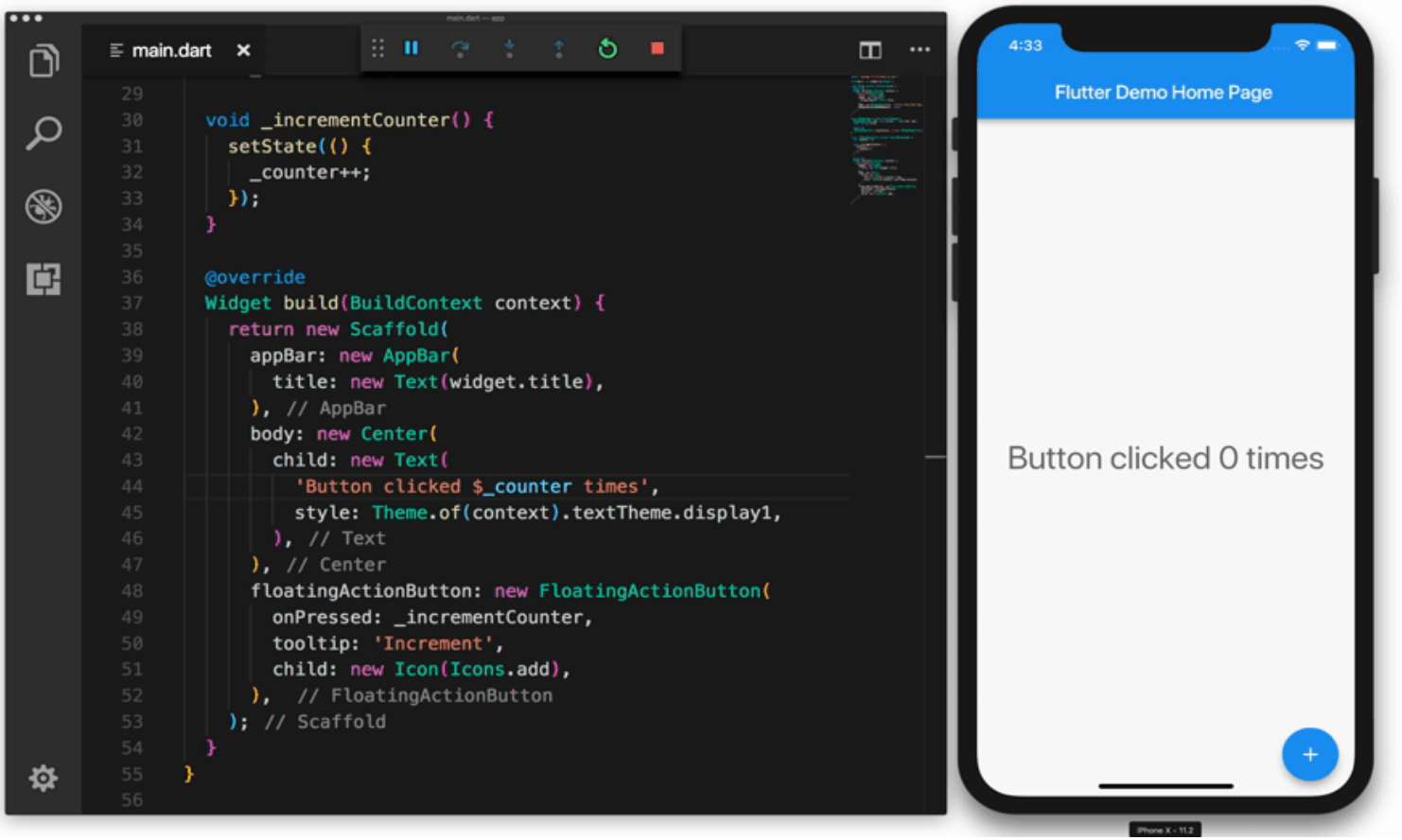
Widgets

- *Stateless vs. Stateful Widgets*
  - *Stateless Widgets*
    - Always built from scratch, when the parent *widget invokes the* `build()` method for its interface to be created
  - *Stateful Widgets*
    - Return a `State<T>` object when instantiated and which is maintained throughout the life cycle of that *widget*
    - It is in the context of the *state object that the* `build()` method is defined so that its interface is created
    - *The lifecycle of the widget can be "followed" (to include some special processing during the evolution in the object lifecycle) overriding several methods that are available in the widget*

Widgets

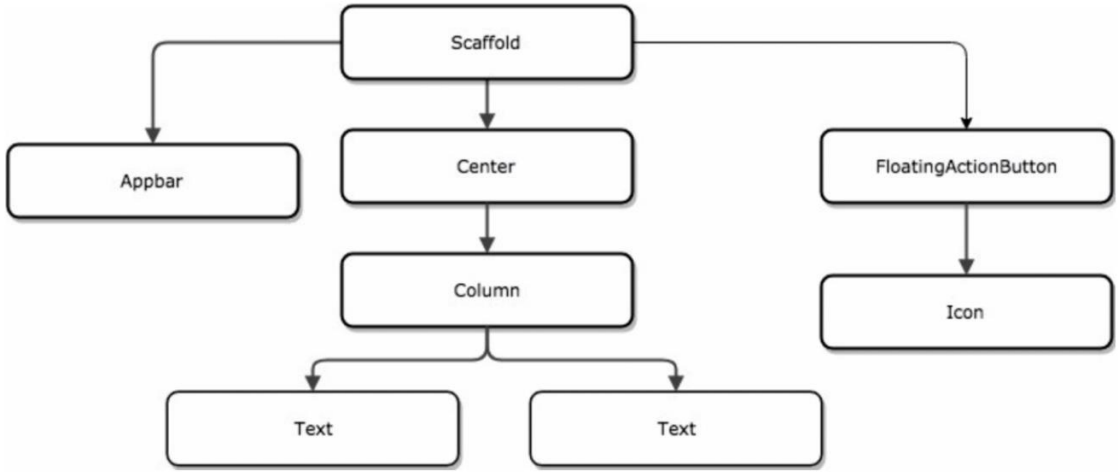

Stateless
Stateful
Constructor
build()
Constructor
Constructor
createState()
didUpdateWidget()
setState()
initState()
build()
build()
build()

Widget tree and hot reload

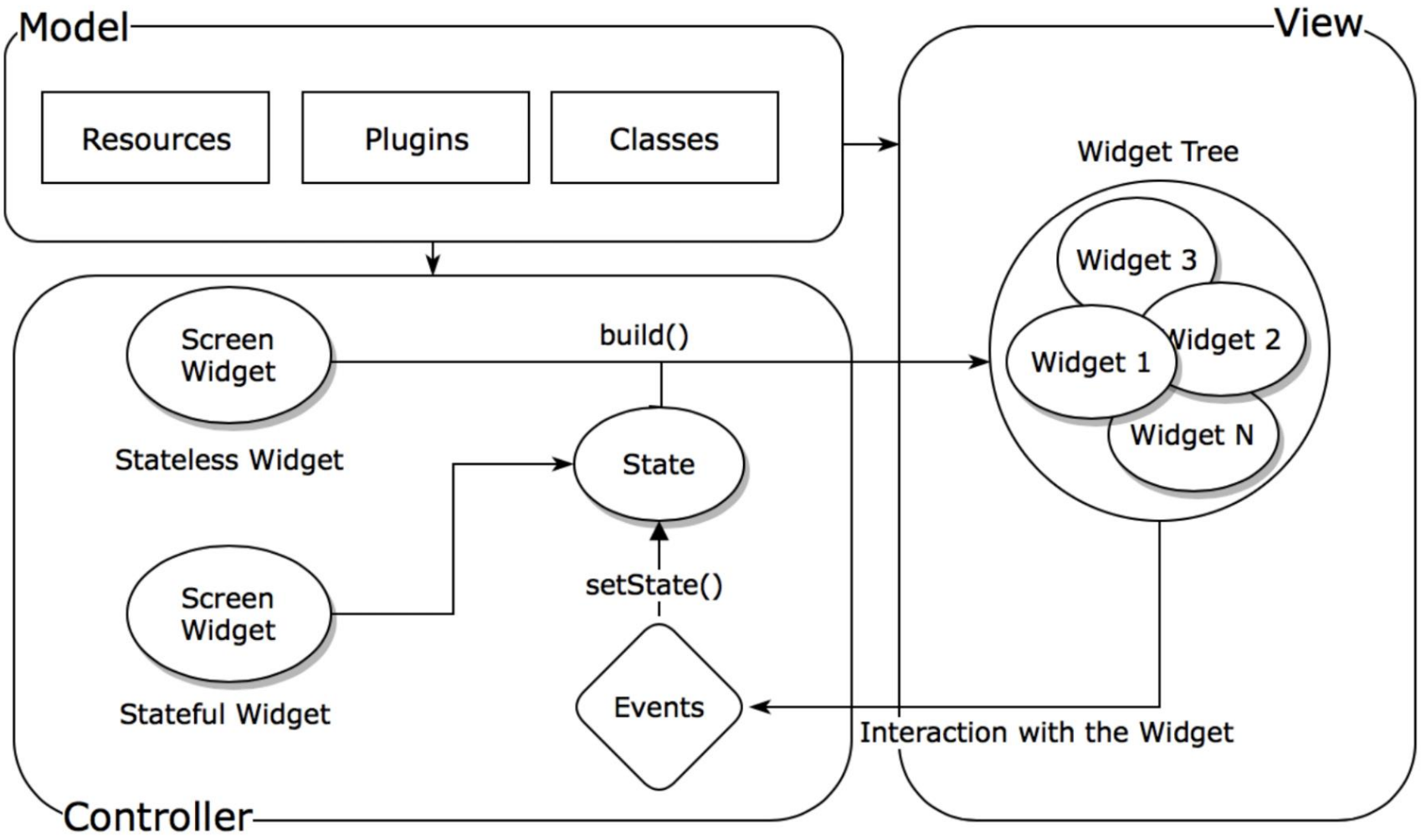


Widget tree


Flutter Demo Home Page
You have pushed the button this many times:
0


Scaffold
Appbar
Center
FloatingActionButton
Column
Icon
Text
Text

State management



State management

- Whenever a UI change/update is desired, the `setState(){}` method must be invoked

- Changed variables related to the state must be placed within `setState`

```
setState(() { _myState = newValue; });
```

- Asynchronous calls should not be made within `setState(){}`
  - If the state is updated after an asynchronous call, it can be called later without values

Installation – Flutter SDK

- Access:
  - https://docs.flutter.dev/get-started/install
- Select the desired OS
- Follow the instructions for each platform
- Run the `flutter doctor -v` command to verify the installation

Installation – IDE

- Various IDEs can be used to develop applications in Flutter

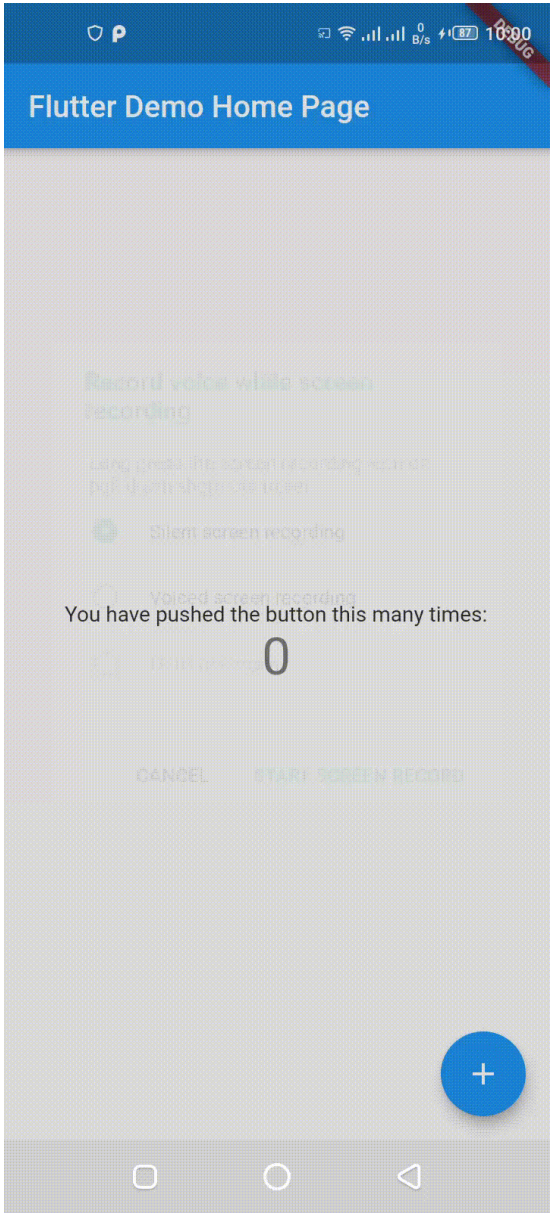
- Most used environments
  - Visual Studio Code
  - Android Studio
  - IntelliJ

- Steps:
  - Access the extension/*plugin* library
  - Search for Flutter
  - Install the extension/*plugin*

Project creation

- Android Studio or IntelliJ
  - `New project` → `Flutter`
- Visual Studio Code
  - Open the command line
  - Execute:

    `flutter create` *`example`*
- An example project is created with a button that allows to increase a value



Structure of a project